Hochschule Furtwangen
Fakultät I
Prof. Dr. Lothar Piepmeyer

Praktikum ‚Objektorientierte Programmierung'

Aufgabenblatt 3

Hier noch einige Vorgaben:

- In keiner der Aufgaben darf der *globale* Namensraum genutzt werden.
- Definieren Sie einen eigenen Namensraum.
- Die Anweisung `using namespace` darf nicht genutzt werden.
- Alle Namen, die in Ihrem Code auftreten sind einfache und klare englische Begriffe. Überhaupt sollte Ihr Code klar und einfach sein. Gehen Sie davon aus, dass es eine einfache und klare Lösung gibt. So sind lange if-else-Kaskaden sicher nicht nötig.

Aufgabe 1:
a. Entwickeln Sie eine Funktion,
`int* copy(int* numbers, int size)`
die ein Array der Länge `size` kopiert. Die Kopie wird auf dem Heap angelegt. Das Ergebnis ist ein Zeiger auf die Kopie. Warum ist der Parameter `size` überhaupt nötig?

b. Entwickeln Sie eine Funktion
`void compare(int* sorted, int* numbers, int size)`
die prüft, ob das *sortierte* Array `sorted` und das Array `numbers` die gleichen Daten enthalten. Genau dann, wenn sie nicht übereinstimmen soll ein assert-failed ausgelöst werden. Das ist schwieriger als es zunächst aussieht. Testen Sie `compare` mit interessanten Testfällen.

c. Mit dem folgenden Code kann man ein Array mit `size` ganzen Zahlen sortieren.

```
void sort(int* numbers, int size){
    for (int i = 1; i < size; ++i) {
        int key = numbers[i];
        int j = i - 1;
        while (j >= 0 && numbers[j] > key) {
            numbers[j + 1] = numbers[j];
            j = j - 1;
        }
        numbers[j + 1] = key;
    }
}
```

Entwickeln Sie eine Funktion

```
void test_sort(int* numbers, int size)
```

die das Array `numbers`, das `size` Elemente enthält, mit Hilfe von `sort` sortiert und testet, ob es wirklich sortiert wurde. Das ist komplizierter als es zunächst aussieht., Nutzen Sie dazu die Aufgabenteile a. und b..

d. Entwickeln Sie eine Funktion

```
void test_sort()
```

die die Funktion aus c. für vier verschiedene Arrays aufruft. Wählen Sie *interessante* Testfälle.

Aufgabe 2:
a. Ersetzen Sie in der Klasse `Position` aus Aufgabenblatt 2 die set-Methode durch einen Konstruktor. Die set-Methode ist nicht mehr erforderlich.

b. Stellen Sie sicher, dass Ihre Klasse den folgenden Test besteht:

```
std::string mordor = "Mordor";
hfu::Position position(mordor, 47, 11);
assert(position.getName() == mordor);
assert(position.getX() == 47);
assert(position.getY() == 11);

mordor[0]='X';
assert(position.getName() == "Mordor");
```

Was ist der Sinn des letzten Tests?

Aufgabe 3:
a. Trennen Sie Klasse `Position` in je eine Datei für Deklaration und Implementierung. Behalten Sie den Namespace bei.

b. Finden Sie experimentell heraus, wie man zwei Objekte vom Datentyp `std::string` vergleichen kann

c. Ergänzen Sie Ihre Klasse um einen Methode `compare` zum sinnvollen Vergleich zweier Objekte vom Typ `Position`. Die folgenden Tests müssen bestanden werden:

```
std::string mordor = "Mordor";
hfu::Position position(mordor, 47, 11);
assert(position.compare(position)==0);
hfu::Position position_name("zordor", 47, 11);
hfu::Position position_x(mordor, 48, 11);
hfu::Position position_y(mordor, 47, 12);
assert(position.compare(position_name)<0);
assert(position.compare(position_x)<0);
assert(position.compare(position_y)<0);
assert(position_name.compare(position)>0);
assert(position_x.compare(position)>0);
assert(position_y.compare(position)>0);
```

Lesen und verstehen Sie den Test-Code, um die Funktionalität von `compare` zu verstehen. Es sollten etwa zehn ordentliche Zeilen reichen, um `compare` zu implementieren.

Aufgabe 4:
Die Methode `compare` lässt sich ausgezeichnet zum Sortieren von Arrays mit Objekten vom

Typ `Position` nutzen. Passen Sie die Funktionen `sort` aus Aufgabe 1 geeignet an. Die Funktion `sort` bleibt dabei eine *Funktion*! Testen Sie!